# Week 8 - Multivariate Analysis

## Please run the cells of the notebook as you get to them while reading

In [1]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import seaborn as sns
from sklearn import decomposition
from sklearn import linear_model
from sklearn import metrics
```

# 1. Lesson: Multivariate graphs

In this lesson, we'll consider some multivariate plots: heatmaps and bubble plots. We'll also consider principal component analysis and multiple regression.

### Heatmaps

In this plot, we use a heatmap to count the total traffic tickets given to citizens. A heatmap requires the x and y axes to be categorical, while the color involves a third, numerical variable. In this example, each row of the DataFrame represents a traffic ticket given to one person in a particular city and year. The data are given as three columns (city, year, and ticket), so we must use a pivot_table to transform it into a two-dimensional grid. The cmap is set to "Blues" in the first example, meaning that the heatmap uses only blue as its color. However, we could also use a multicolor map like "viridis" in the second example. I personally think "Blues" is easier to read - what do you think? Here is a reference guide to the color maps:

https://matplotlib.org/stable/gallery/color/colormap_reference.html

In [2]:
```
np.random.seed(0)
num_samples = 100
city_series = np.random.choice(["New York", "Boston", "Atlanta", "Wash
year_series = np.random.choice([2018, 2019, 2020], size = num_samples)
ticket_series = np.random.randint(50, 150, size = num_samples) # Traff
df = pd.DataFrame({"city": city_series, "year": year_series, "ticket":
pivot = df.pivot_table(index = ["city"], columns = ["year"], values =
```

In [3]:
```
df.iloc[0:5] # first five rows of a long DataFrame
```

Out[3]:

| | city | year | ticket |
|---|---|---|---|
| **0** | New York | 2019 | 79 |
| **1** | Washington, D. C. | 2020 | 71 |
| **2** | Boston | 2018 | 75 |
| **3** | New York | 2018 | 130 |
| **4** | Washington, D. C. | 2020 | 110 |

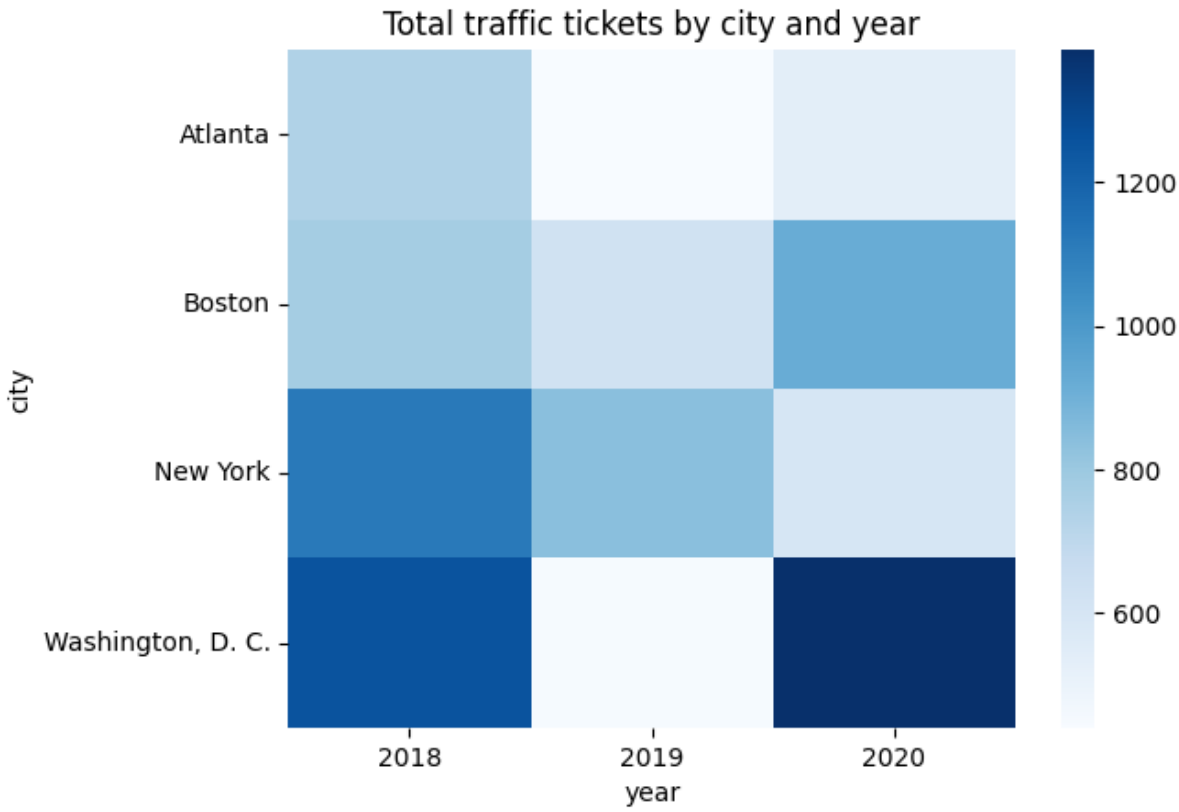
In [4]:
```
pivot # 2 by 2 grid showing total ticket amounts
```

Out[4]:

| year | 2018 | 2019 | 2020 |
|---|---|---|---|
| **city** | | | |
| **Atlanta** | 742 | 440 | 538 |
| **Boston** | 775 | 628 | 922 |
| **New York** | 1122 | 841 | 596 |
| **Washington, D. C.** | 1258 | 451 | 1385 |

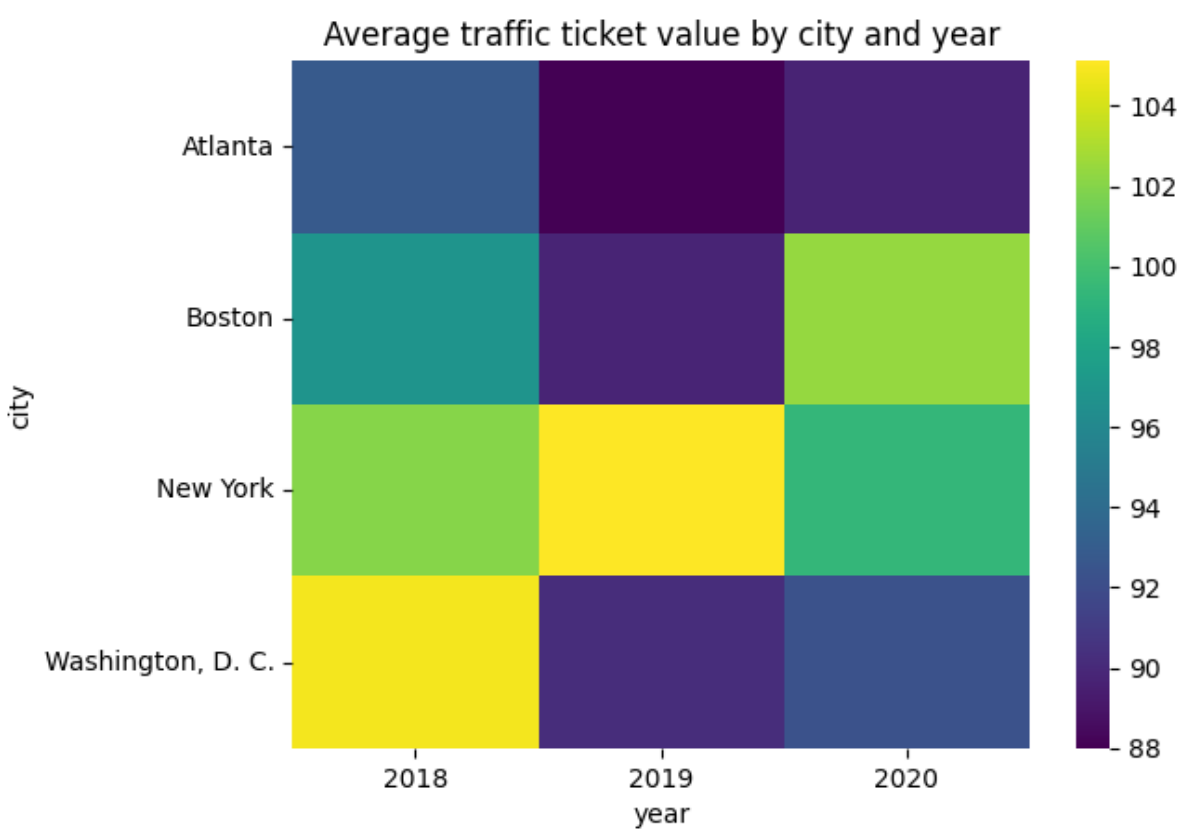
In [5]:
```
ax = sns.heatmap(pivot, cmap = "Blues")
ax.set_title("Total traffic tickets by city and year")
```

Out[5]:
```
Text(0.5, 1.0, 'Total traffic tickets by city and year')
```




In [6]:
```
pivot2 = df.pivot_table(index = ["city"], columns = ["year"], values =
ax2 = sns.heatmap(pivot2, cmap = "viridis") # do you think this is a b
ax2.set_title("Average traffic ticket value by city and year")
```

Out[6]: Text(0.5, 1.0, 'Average traffic ticket value by city and year')

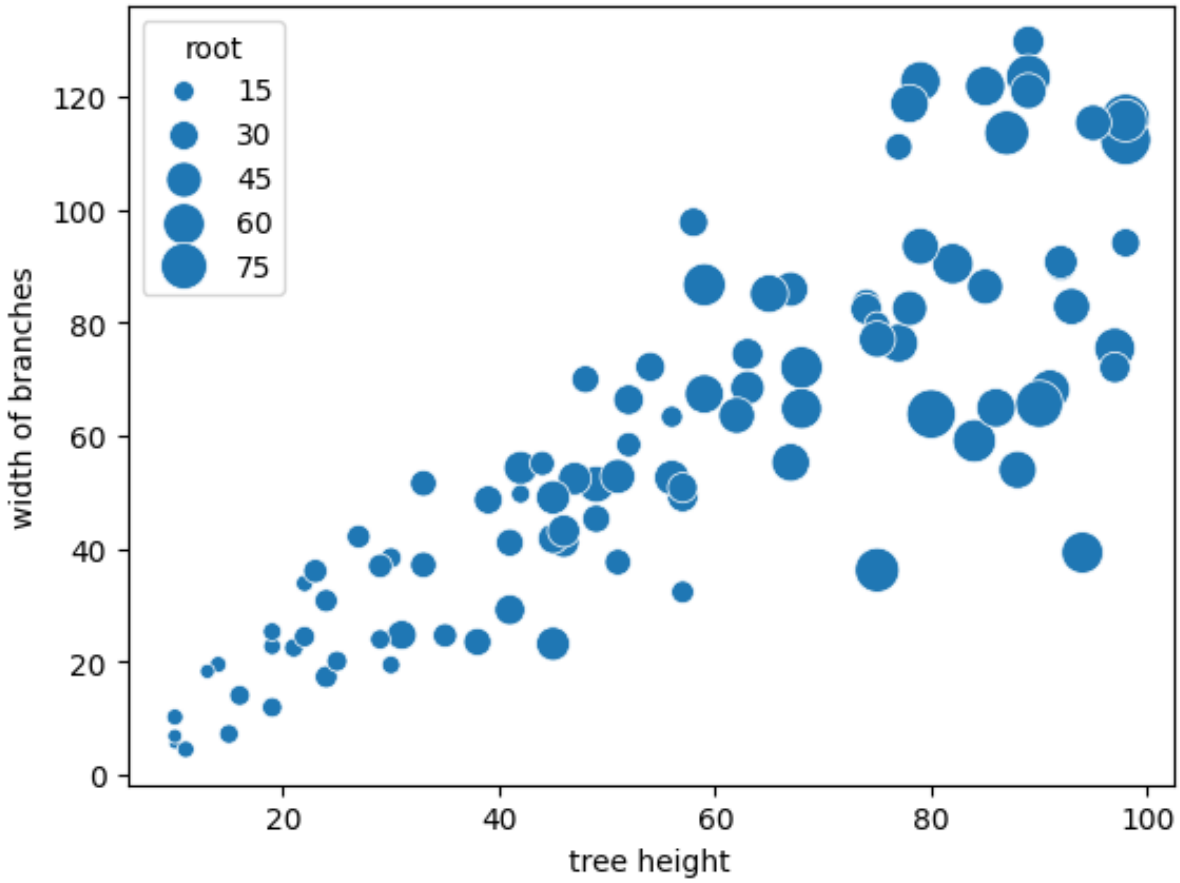


## Bubble plots

A bubble plot is just a scatter plot, but with the dot size representing a third variable. Here, each data point shows a tree (I mean the kind of tree that is a plant and grows in the ground, not a decision tree), with its height, width of branches, and root depth. The bubble size in the bubble plot represents the root depth, while the other variables are shown on the x and y axes. From this plot, we can see that the root depth is strongly correlated with both the width of branches and the tree height.

In [7]:
```
np.random.seed(0)
num_samples = 100
height_series = np.random.randint(10, 100, size = num_samples) # tree
width_series = height_series * np.maximum(np.random.normal(scale = 0.3
root_series = height_series * np.maximum(np.random.normal(scale = 0.2,
df2 = pd.DataFrame({'height': height_series, 'width': width_series, 'r
minroot = root_series.min()
maxroot = root_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df2, x = "height", y = "width", size = "ro
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
```

Out[7]:
```
Text(0, 0.5, 'width of branches')
```




## Principal Component Analysis

Principal Component Analysis can be thought of as a way to use one (or a few) linear combinations of the coordinates in order to explain all of the coordinates. It is most useful when there are strong correlations between some or all of the coordinates. Principal Component Analysis can show us which linear combinations are most (and least) useful for explaining the data.

With the trees data, the three values (height, width, and root) are highly correlated. For this reason, Principal Component Analysis shows that most of the variance can be explained by a single value which is a linear combination of height, width, and root. What this means is that if the first components_ vector in the code below (the "first principal component") is [a, b, c], for some specific numbers a, b, and c, then the value v = a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r), where the mu's are the means, is highly informative about the values of height, width, and root. In this case, because height, width, and root are so highly correlated, we can almost assume that all of the data is located along an axis defined by the vector [a, b, c]. The point v * [a, b, c] + (mean vector) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (mean vector) is therefore approximately equal to the data point (height, width, root) in

question. The mean vector is [mu_h, mu_w, mu_r]. Importantly, there is still variance unexplained by this one value - just knowing v * [a, b, c] does not tell you the height, width, and root with 100% accuracy. If the second principal component is [d, e, f], then we could approximate the tree vector even more accurately as: (height, width, root) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (d * (height - mu_h) + e * (width - mu_w) + f * (root - mu_r)) * [d, e, f] + (mean vector).

The printing of "components_", below, shows the first principal component [a, b, c]. If you draw a vector in the direction shown, you will be heading in the direction where the values height, width, and root change most rapidly. A later computation shows all three principal component vectors.

**Optional exercise for the reader:** show that if all three principal components are included, then equality is exact. This is going to be true simply because the principal components are orthonormal: the projections onto a complete set of orthonormal vectors always add up to the original vector.

(height, width, root) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (d * (height - mu_h) + e * (width - mu_w) + f * (root - mu_r)) * [d, e, f] + (g * (height - mu_h) + h * (width - mu_w) + i * (root - mu_r)) * [g, h, i] + [mu_h, mu_w, mu_r].

For this proof, we must use the fact that the principal component vectors are orthonormal. That is, any pair of them are orthogonal, even with a transpose (ad + be + cf = 0, ag + bh + ci = 0, dg + eh + fi = 0, ab + de + gh = 0, ac + df + gi = 0, bc + ef + hi = 0) and any one of them is a unit vector, even with a transpose (a^2 + b^c + c^2 = 1, d^2 + e^2 + f^2 = 1, g^2 + h^2 + i^2 = 1, a^2 + d^2 + g^2 = 1, b^2 + e^2 + h^2 = 1, c^2 + f^2 + i^2 = 1). If you write out the equation above as a single vector, then apply the orthonormality rules, you should be able to prove equality.

In [8]:
```
pca = decomposition.PCA(n_components = 1)
pca.fit(df2)
print(f"The first principal component explains {pca.explained_variance
print(f"The components_ vector is [a, b, c] = {pca.components_}")
mu_h, mu_w, mu_r = df2.mean().values
print(f"(height, width, root) is approximately equal to ({pca.componen
print("Examples: (estimating the the vector equals its projection onto
df_example = df2.copy()
df_example["v"] = pca.components_[0][0] * (df2["height"] - mu_h) + pca
df_example["height_est"] = df_example["v"] * pca.components_[0][0] + m
df_example["width_est"] = df_example["v"] * pca.components_[0][1] + mu
df_example["root_est"] = df_example["v"] * pca.components_[0][2] + mu_
print(df_example.iloc[0:5])
```

```
print("Means:")
print(f"mu_h = {mu_h:.3}", f"mu_w = {mu_w:.3}", f"mu_r = {mu_r:.3}")
```

```
The first principal component explains 0.8686108480944689 of the varian
ce
The components_ vector is [a, b, c] = [[0.59031629 0.71908728 0.3666608
3]]
(height, width, root) is approximately equal to (0.59 * (height - 5.6e+
01) + 0.72 * (width - 5.9e+01) + 0.37 * (root - 3.6e+01)) * (0.59, 0.7
2, 0.37) + (5.6e+01, 5.9e+01, 3.6e+01)
Examples: (estimating the the vector equals its projection onto the fir
st principal component)
   height       width       root          v  height_est  width_est   ro
ot_est
0      54   72.139334  31.517932   6.287751   60.001762  63.563941  38.
673095
1      57   49.169527  33.768693  -7.633330   51.783921  53.553468  33.
568780
2      74   83.443733  28.770850  25.215680   71.175227  77.174773  45.
613225
3      77   76.353866  53.061241  30.794730   74.468631  81.186597  47.
658844
4      77  111.095365  26.006307  45.856916   83.360084  92.017624  53.
181557
Means:
mu_h = 56.3 mu_w = 59.0 mu_r = 36.4
```

If we were to choose a second or third component to preserve, that values below show that each would explain a small, additional amount of the variance. This means that with this particular dataset, we should likely do PCA using only one component and not two. If the components' explained variances were closer in magnitude to each other, we might have to include two or even all three values to explain the data.

In [9]:
```
pca3 = decomposition.PCA(n_components = 3)
pca3.fit(df2)
print(pca3.explained_variance_ratio_)
print(pca3.components_)
```

```
[0.86861085 0.09414617 0.03724298]
[[ 0.59031629  0.71908728  0.36666083]
 [ 0.32633701 -0.62809127  0.70640322]
 [ 0.73826204 -0.29734633 -0.6054373 ]]
```

The principal component vectors shown are orthogonal to each other (their dot product is basically zero). Here, we check that using the first two vectors.

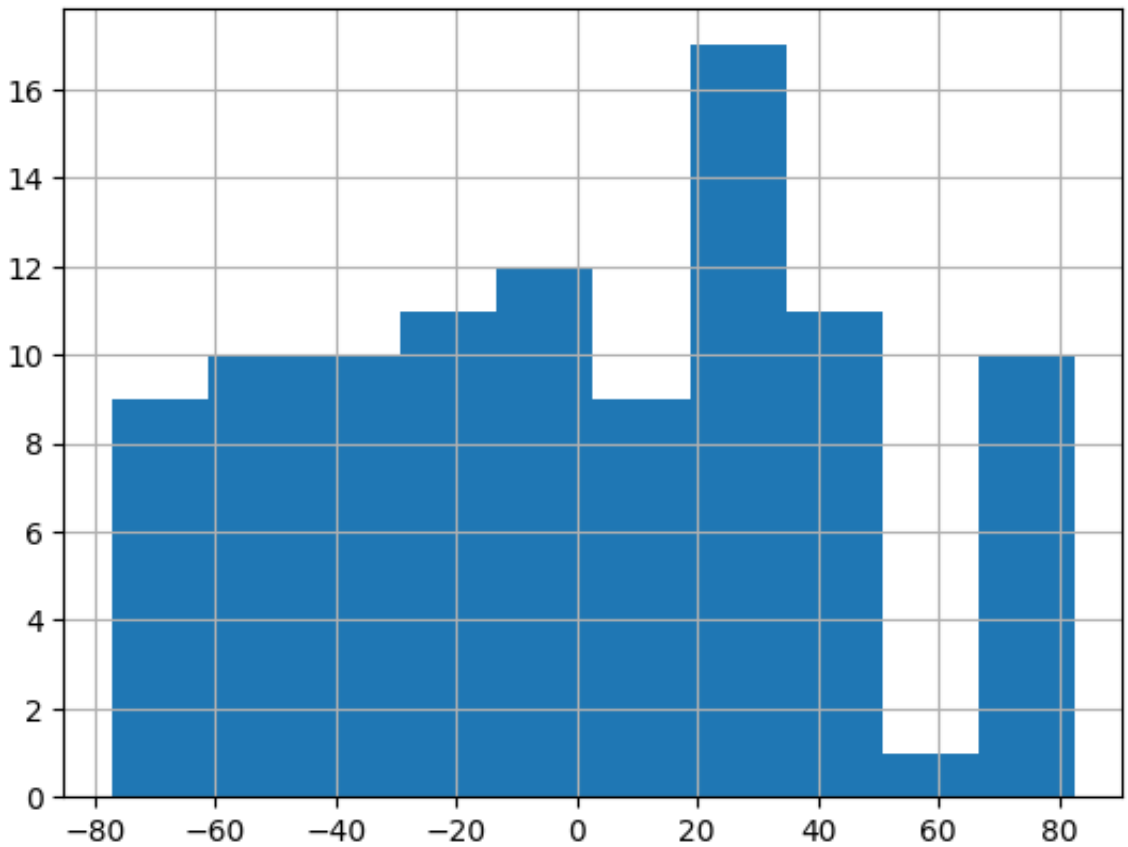
In [10]:
```
print(np.array(pca3.components_[0]).dot(np.array(pca3.components_[1]))
```

```
-1.6653345369377348e-16
```

We can use the projection onto the first principal component as if it were a new feature. For instance, we could plot a histogram of the data against the magnitude of the projection onto the first principal component. In fact, for some purposes, this magnitude (which we have called v) might make a better feature for data analysis than either height, width, or root. We must subtract off the mean before the dot product; the understanding is that this represents the component of the data relative to the mean.

In [11]:
```
((df2 - df2.mean()).dot(np.array(pca.components_[0]))).hist()
```

Out[11]:
```
<Axes: >
```




In [12]:
```
# Showing that this is the same as the value we've called "v"
(df_example["v"] - (df2 - df2.mean()).dot(np.array(pca.components_[0])
```

Out[12]:
```
np.float64(0.0)
```

## Linear Regression

Now let's run a linear regression model to predict the root from the height and width, as well as predicting the third principal component from the first and second. Which is easier to predict?

### Predicting the root from the height and width

In [13]:
```
linreg = linear_model.LinearRegression()
linreg.fit(df2[['height', 'width']], df2['root'])
preds = linreg.predict(df2[['height', 'width']])
metrics.mean_squared_error(preds, df2['root']) # Mean squared error as
```

Out[13]:
```
138.22487356329611
```

In [14]:
```
np.corrcoef(preds, df2['root']) # Correlation coefficient as a metric
```

Out[14]:
```
array([[1.        , 0.79468295],
       [0.79468295, 1.        ]])
```

### Predicting the third principal component from the first and second

In [15]:
```
first_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3
second_principal_component_score = (df2 - df2.mean()).dot(np.array(pca
third_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3
```

In [16]:
```
linreg_first = linear_model.LinearRegression()
pc_X = np.array([first_principal_component_score, second_principal_com
linreg_first.fit(pc_X, third_principal_component_score)
preds = linreg_first.predict(pc_X)
metrics.mean_squared_error(preds, third_principal_component_score) # T
```

Out[16]:
```
78.74928905273173
```

In [17]:
```
np.corrcoef(preds, third_principal_component_score) # More importantly
```

Out[17]:
```
array([[1.00000000e+00, 1.55495941e-14],
       [1.55495941e-14, 1.00000000e+00]])
```

Here, we see that the projections of the first two principal components have no ability to predict the projection of the third, when measured by correlation. This is a special property of principal components. It is intended to ensure that the each component gives new information not present in the previous components.

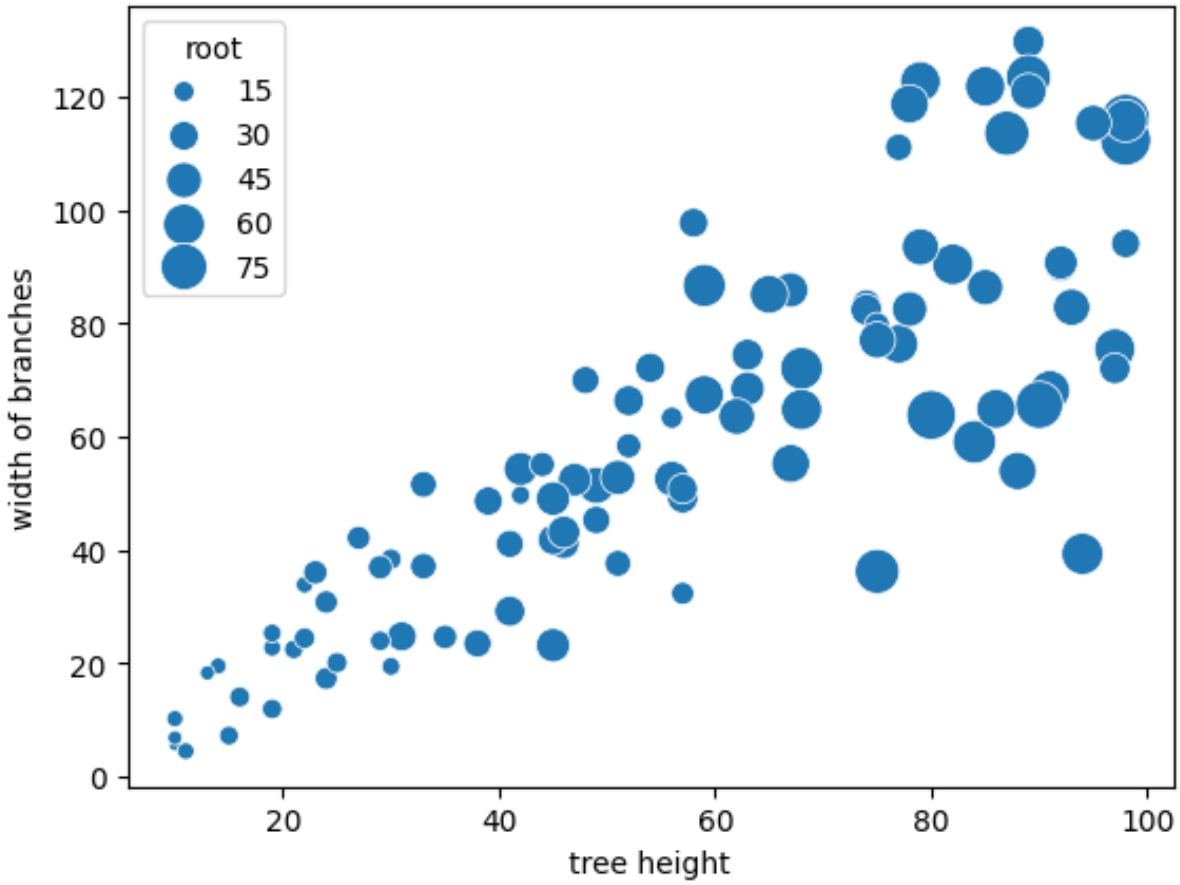
The mean_squared_error is misleading - it makes it look like the error is less for the second computation than the first; but the correlation coefficient is giving more useful information here. We cannot compare the mean squared error for two totally different computations, because mean squared error is highly dependent on the magnitudes of the values in question!

# 2. Weekly graph question

The tree data above can be graphed via either a pairplot or a bubble plot. What are the advantages and disadvantages of each?

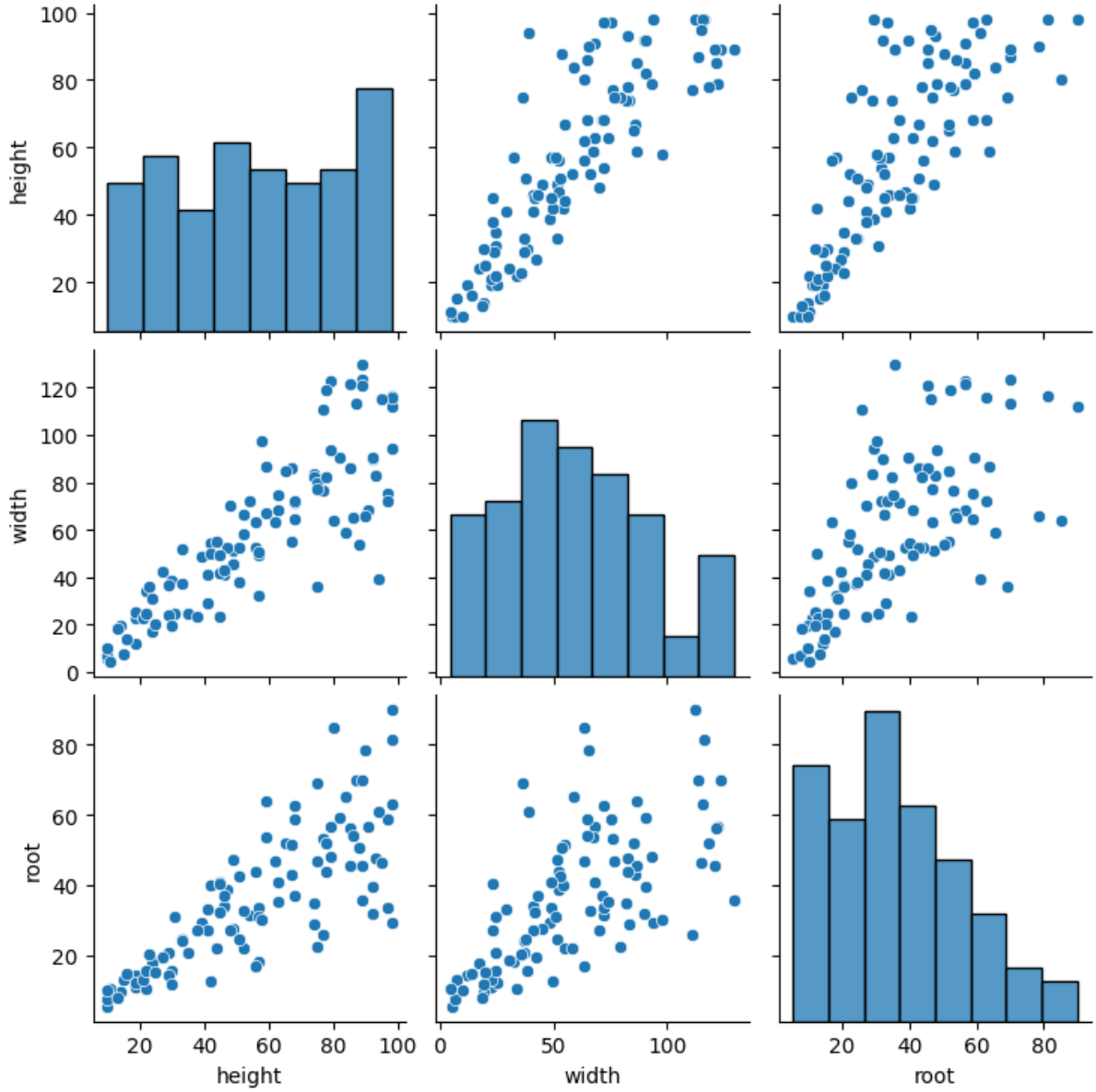
In [18]:
```
ax = sns.scatterplot(data = df2, x = "height", y = "width", size = "ro
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
```

Out[18]:
```
Text(0, 0.5, 'width of branches')
```




In [19]:
```
sns.pairplot(df2)
```

Out[19]:
```
<seaborn.axisgrid.PairGrid at 0x23df606d1e0>
```




## Pair Plot:

Pros:

- The pair plot includes pairwise relationships between each of the features. It allows us to see histograms of a single feature, and also how the combinations with other features look.

Cons:

- The pair plot does not extend to comparing three features at once. With the bubble plot you can visualize all three together, but the pair plot you need to look at two combinations A,B and B,C and try to extrapolate out to how A,B,C would combine. The other downside is that the pair plot becomes crowded as the number of features grows

### Bubble Plot:

Pros:

- The bubble plot visualizes a single relationship between three features at once. In this case, it is helpful to understand at a glance that the feature "root" is correlated with both "width" and "height".

Cons:

- The bubble plot does not include the histograms of each feature, so you would most likely need to plot them separately.

# 3. Homework - Grocery Store Correlation

In a particular state, the price of milk is highly correlated with the price of cheese. Neither is correlated with the price of bread.

In [20]:
```
num_points = 1000
np.random.seed(0)
milk_series = np.maximum(np.random.normal(scale = 0.5, loc = 2.00, siz
cheese_series = milk_series * np.maximum(np.random.normal(scale = 0.1,
bread_series = np.maximum(np.random.normal(scale = 1.0, loc = 3.00, si
df3 = pd.DataFrame({"milk": milk_series, "cheese": cheese_series, "bre
```

In [21]:
```
df3
```

Out[21]:

| | milk | cheese | bread |
|---|---|---|---|
| **0** | 2.882026 | 3.042256 | 1.467079 |
| **1** | 2.200079 | 2.396430 | 1.288030 |
| **2** | 2.489369 | 2.384239 | 3.046135 |
| **3** | 3.120447 | 3.153122 | 2.041626 |
| **4** | 2.933779 | 3.000685 | 2.919188 |
| **...** | ... | ... | ... |
| **995** | 2.206435 | 2.228003 | 5.079177 |
| **996** | 1.900801 | 2.167202 | 2.092534 |
| **997** | 2.047096 | 2.079529 | 2.807596 |
| **998** | 1.426195 | 1.263337 | 1.787484 |
| **999** | 1.820943 | 1.582223 | 2.919401 |

1000 rows × 3 columns

## Heatmap

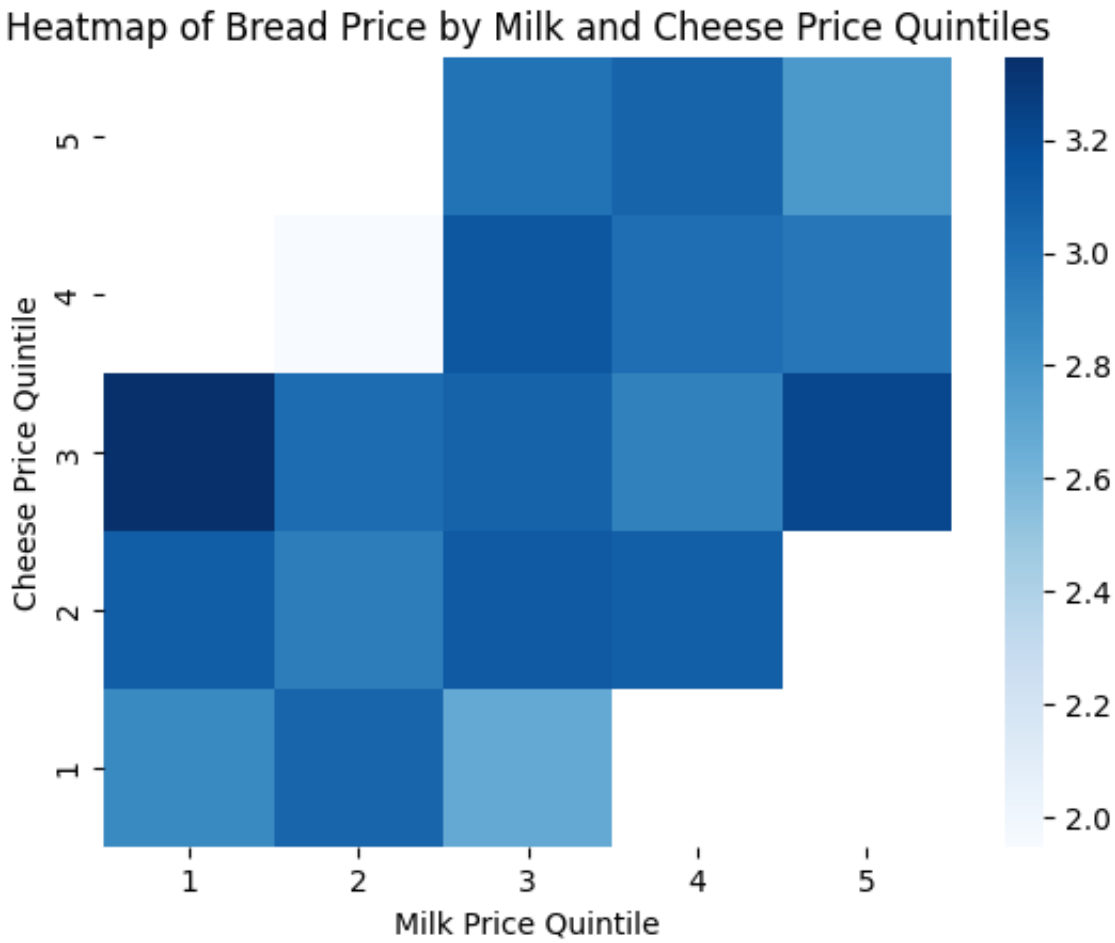
Draw a heatmap where the x-axis category is a discretized version of the milk price and the y-axis category is a discretized version of the cheese price. That is, you can choose "low", "medium", and "high" for the milk price, or, if you prefer, you can choose 1, 2, 3, 4, and 5 for the five quintiles. (The first quintile should include the bottom fifth of the data, the second quintile contains the next fifth, and so on).

Try Googling "how do I change numerical to categorical data" to get advice on this.

In [22]:
```
import matplotlib.pyplot as plt
```

In [23]:
```
df3['milk_bin'] = pd.qcut(df3['milk'], 5, labels=[1, 2, 3, 4, 5])
df3['cheese_bin'] = pd.qcut(df3['cheese'], 5, labels=[1, 2, 3, 4, 5])

df3_pivot = df3.pivot_table(index='cheese_bin', columns='milk_bin', va

sns.heatmap(df3_pivot, cmap='Blues')
plt.title('Heatmap of Bread Price by Milk and Cheese Price Quintiles')
plt.xlabel('Milk Price Quintile')
plt.ylabel('Cheese Price Quintile')
plt.gca().invert_yaxis()
None
```

```
C:\Users\caleb\AppData\Local\Temp\ipykernel_7388\3251682398.py:4: Futur
eWarning: The default value of observed=False is deprecated and will ch
ange to observed=True in a future version of pandas. Specify observed=F
alse to silence this warning and retain the current behavior
  df3_pivot = df3.pivot_table(index='cheese_bin', columns='milk_bin', v
alues='bread', aggfunc='mean')
```

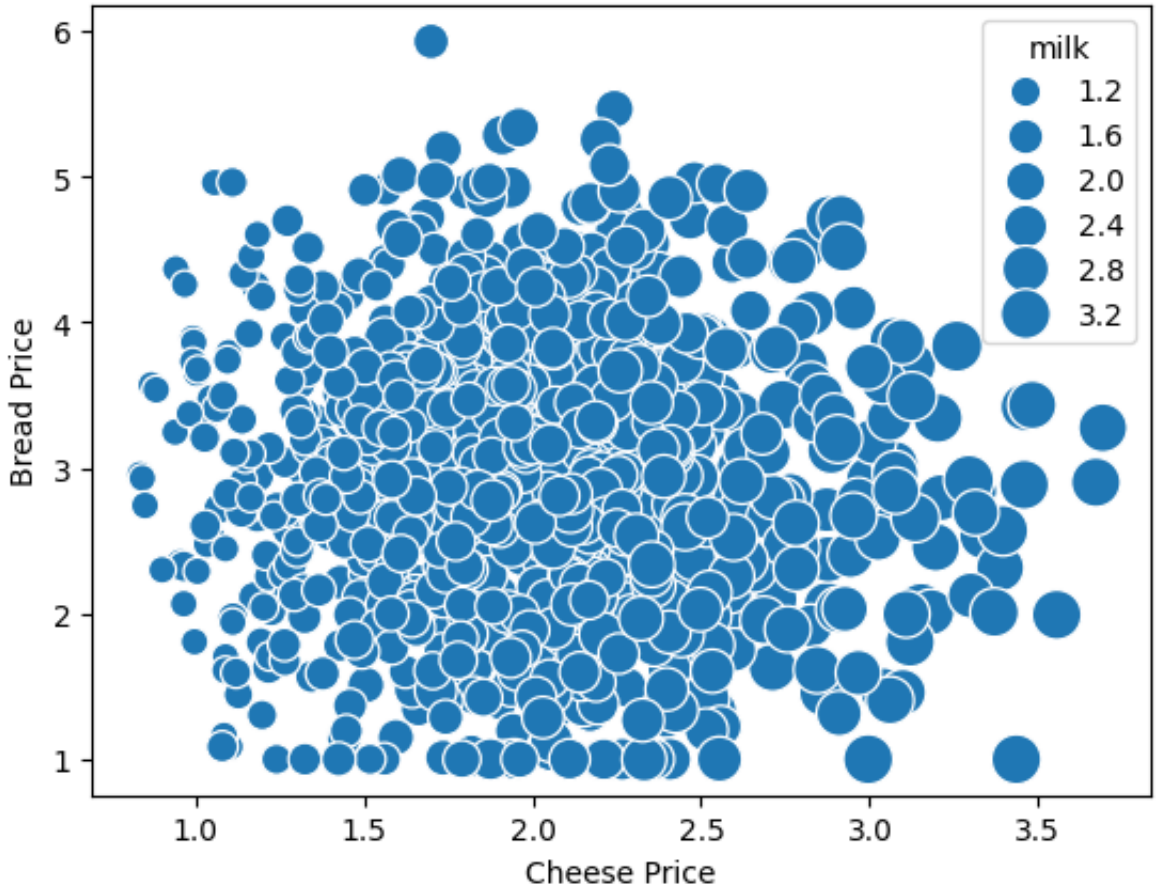



## Bubble plot

Try all three possible bubble plots: one where the size of the dot is determined by the milk price, one where it's the cheese price, and one where it's the bread price. Which of these plots is / are the most informative and useful?

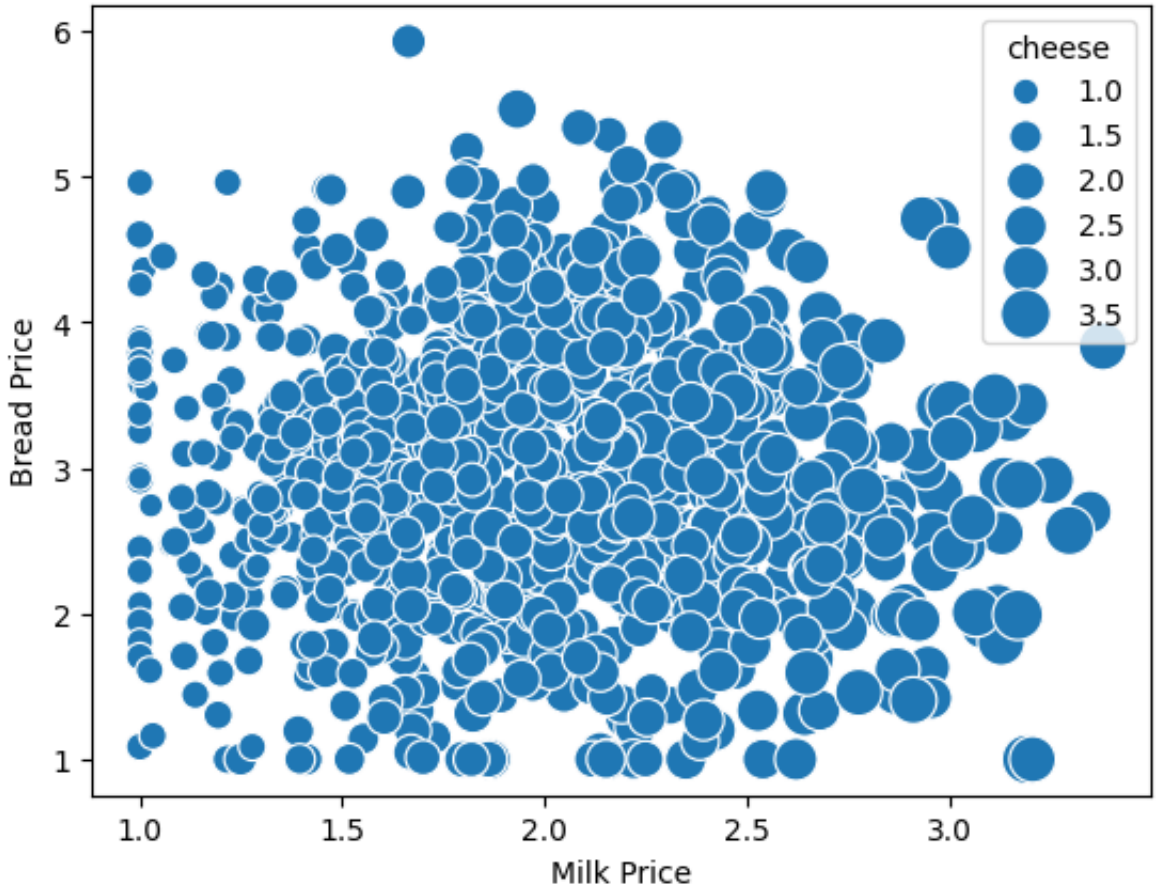
In [24]:
```
# Dot is milk price
minroot = milk_series.min()
maxroot = milk_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df3, x = "cheese", y = "bread", size = "mi
ax.set_xlabel("Cheese Price")
ax.set_ylabel("Bread Price")
```

Out[24]:
```
Text(0, 0.5, 'Bread Price')
```



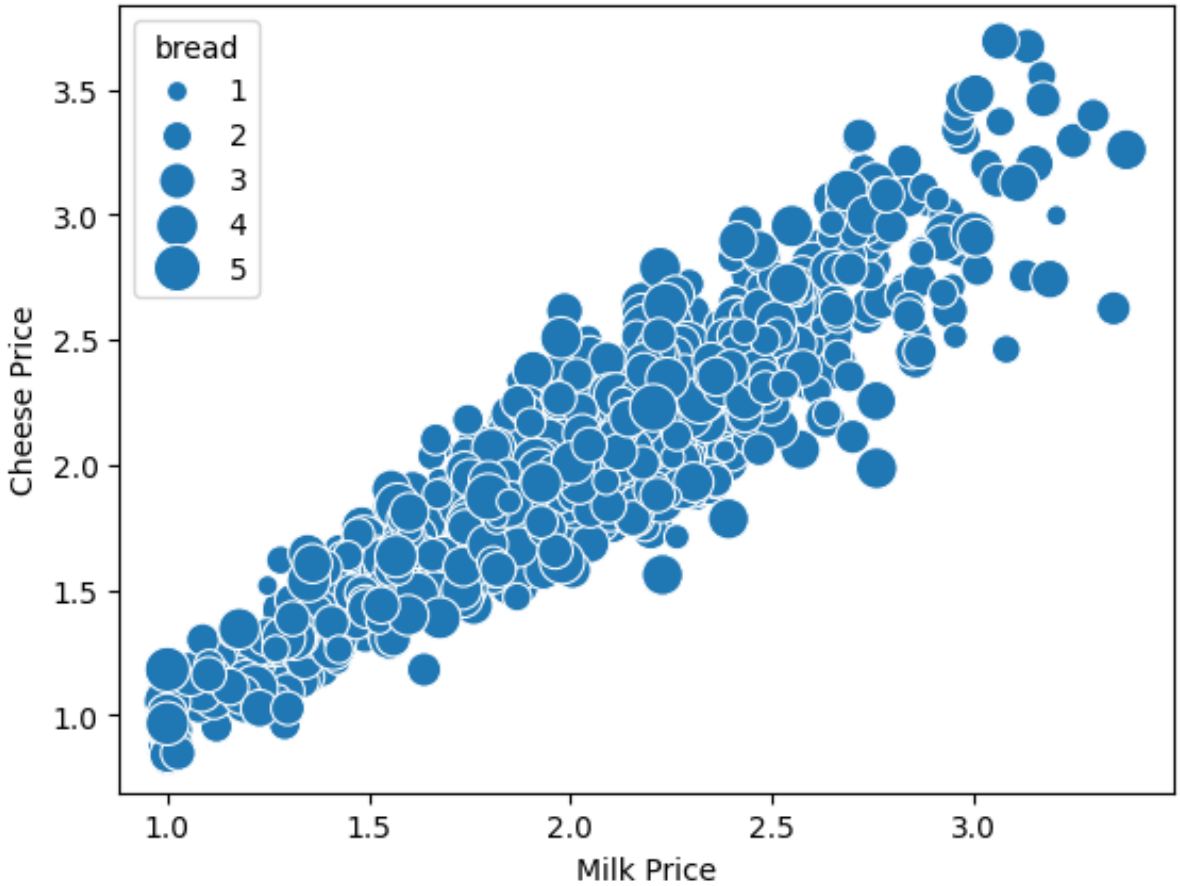

In [25]:
```
# Dot is Cheese price
minroot = cheese_series.min()
maxroot = cheese_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df3, x = "milk", y = "bread", size = "chee
ax.set_xlabel("Milk Price")
ax.set_ylabel("Bread Price")
```

Out[25]: Text(0, 0.5, 'Bread Price')




In [26]:
```
# Dot is Bread price
minroot = bread_series.min()
maxroot = bread_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df3, x = "milk", y = "cheese", size = "bre
ax.set_xlabel("Milk Price")
ax.set_ylabel("Cheese Price")
```

Out[26]:
```
Text(0, 0.5, 'Cheese Price')
```




The most informative plot is the bubble plot with the Bread Price as the dot size. The other two versions of the plot are a cloud of noise with no patterns. We can see a strong correlation between the Milk and Cheese prices, and do not see a strong correlation between those and the Bread Price due to the dot size looking random.

## Principal component analysis

Perform a principal component analysis of the data. What are the first, second, and third component vectors?

Challenge question: explain why the three vectors point as they do. (Could someone have guessed the order of the three vectors, even before running the analysis?)

In [27]:
```
pca = decomposition.PCA(n_components = 3)
pca.fit(df3)

print(f"The first principal component explains {pca.explained_variance
print(f"The second principal component explains {pca.explained_varianc
print(f"The third principal component explains {pca.explained_variance
print(f"The components_ vector is {pca.components_}")
```

```
The first principal component explains 0.7829444272057101 of the varian
ce
The second principal component explains 0.16324412274814487 of the vari
ance
The third principal component explains 0.04522838893323079 of the varia
nce
The components_ vector is [[ 0.22299336  0.23834686 -0.01069345  0.6683
7654  0.66829873]
 [-0.00740863 -0.00431591  0.99983153  0.00438285  0.01562627]
 [ 0.15322251 -0.14385202  0.00828586  0.69140406 -0.6911738 ]]
```

Since the Milk and Cheese prices are so highly correlated, the first component can explain the majority of the variance of both. The second component likely explains the Bread price variance and the final component the remaining variance from the Milk or Cheese prices.

## Linear regression

Use linear regression to predict the milk price from the other two, the cheese price from the other two, and the bread price from the other two. Which prediction(s) work the best, and why?

In [28]:
```
# Predict Milk Price
linreg = linear_model.LinearRegression()
linreg.fit(df3[['cheese', 'bread']], df3['milk'])
preds = linreg.predict(df3[['cheese', 'bread']])
metrics.mean_squared_error(preds, df3['milk']) # Mean squared error as
```

Out[28]: 0.03447410921882385

In [29]:
```
# Predict Cheese Price
linreg = linear_model.LinearRegression()
linreg.fit(df3[['milk', 'bread']], df3['cheese'])
preds = linreg.predict(df3[['milk', 'bread']])
metrics.mean_squared_error(preds, df3['cheese']) # Mean squared error
```

Out[29]: 0.0396561665666383

In [30]:
```
# Predict Bread Price
linreg = linear_model.LinearRegression()
linreg.fit(df3[['milk', 'cheese']], df3['bread'])
preds = linreg.predict(df3[['milk', 'cheese']])
metrics.mean_squared_error(preds, df3['bread']) # Mean squared error a
```

Out[30]: 0.8772889513306434

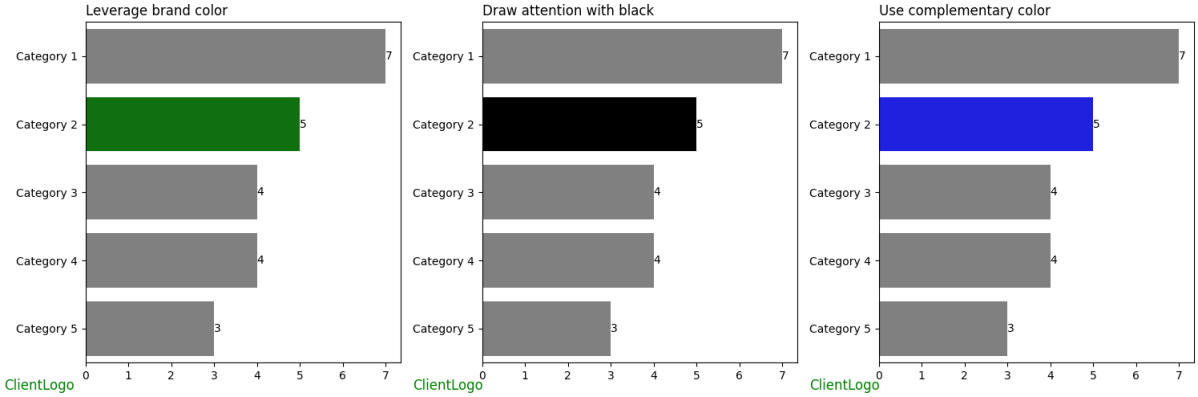
Predicting Milk from the Cheese and Bread price works the best, closely followed by predicting Cheese from the Milk and Bread price. We know that the Milk and

Cheese price vary together, so they can be used to predict each other.

# 4. Storytelling With Data plot

Reproduce any graph of your choice in p. 120-135 of the Storytelling With Data book as best you can. ("Use color consistently" in chapter four up to "Create a clear visual hierarchy of information" in chapter five.) You do not have to get the exact data values right, just the overall look and feel.

In [87]:
```
# Figure 4.16 (Page 124)
df_4 = pd.DataFrame(
    [[7,5,4,4,3]],
    columns=['Category 1', 'Category 2', 'Category 3', 'Category 4', '
)

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 5), sharey=True

first_colors = ['gray' if col != 'Category 2' else 'green' for col in
second_colors = ['gray' if col != 'Category 2' else 'black' for col in
third_colors = ['gray' if col != 'Category 2' else 'blue' for col in d

sns.barplot(data=df_4, orient = 'h', palette=first_colors, ax=ax1)
ax1.set_title("Leverage brand color", loc='left')
plt.figtext(0.0, 0.01, "ClientLogo", fontsize=12, color="green")

sns.barplot(data=df_4, orient = 'h', palette=second_colors, ax=ax2)
ax2.set_title("Draw attention with black", loc='left')
plt.figtext(0.34, 0.01, "ClientLogo", fontsize=12, color="green")

sns.barplot(data=df_4, orient = 'h', palette=third_colors, ax=ax3)
ax3.set_title("Use complementary color", loc='left')
plt.figtext(0.67, 0.01, "ClientLogo", fontsize=12, color="green")

for container in ax1.containers:
    ax1.bar_label(container)
for container in ax2.containers:
    ax2.bar_label(container)
for container in ax3.containers:
    ax3.bar_label(container)
ax1.tick_params(labelleft=True)
ax2.tick_params(labelleft=True)
ax3.tick_params(labelleft=True)
plt.tight_layout()
```


Leverage brand color
Category 1
Category 2
Category 3
Category 4
Category 5
7
5
4
4
3
0 1 2 3 4 5 6 7
ClientLogo
Draw attention with black
Category 1
Category 2
Category 3
Category 4
Category 5
7
5
4
4
3
0 1 2 3 4 5 6 7
ClientLogo
Use complementary color
Category 1
Category 2
Category 3
Category 4
Category 5
7
5
4
4
3
0 1 2 3 4 5 6 7
ClientLogo